Multilevel Inheritance in Java

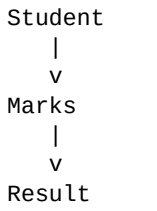
Java Programming Lab Record — Multiple Test Cases

1. Aim

To implement Multilevel Inheritance in Java and execute the program using multiple test cases to verify the results.

2. Concept

In multilevel inheritance, one class inherits from another class, and a third class inherits from the second class.



- **Student** → Parent class
- **Marks** → Child of Student
- **Result** → Child of Marks

3. Java Program

```
import java.util.Scanner;

// Level 1 - Parent class
class Student {
    String name;
    int rollNo;

    void getStudentDetails(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }
}

// Level 2 - Inherits Student
class Marks extends Student {
    int marks;

    void getMarks(int marks) {
        this.marks = marks;
    }
}

// Level 3 - Inherits Marks
class Result extends Marks {

    void displayResult() {
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Marks: " + marks);

        if (marks >= 35) {
            System.out.println("Result: Pass");
        } else {
            System.out.println("Result: Fail");
```

```
        }
    }
}

// Main class
public class MultilevelInheritance {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of test cases: ");
        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 1; i <= n; i++) {

            System.out.println("\n----- Test Case " + i + " -----");

            System.out.print("Enter student name: ");
            String name = sc.nextLine();

            System.out.print("Enter roll number: ");
            int rollNo = sc.nextInt();

            System.out.print("Enter marks: ");
            int marks = sc.nextInt();
            sc.nextLine();

            // Creating object of the last class
            Result r = new Result();

            // Calling methods from all three levels
            r.getStudentDetails(name, rollNo);
            r.getMarks(marks);
            r.displayResult();
        }

        sc.close();
    }
}
```

4. Explanation

• **Student** is the top-level parent class, holding name and roll number.

• **Marks** extends Student, adding the marks field — this is the second level.

• **Result** extends Marks, so it inherits fields and methods from both Marks and Student — this is the third level.

• A single `Result` object can call `getStudentDetails()`, `getMarks()`, and `displayResult()` even though only the last method is defined directly in Result.

• The program reads the number of test cases first, then loops that many times, creating a fresh Result object for each student.

5. Test Case Verification

Test Case	Name	Roll No.	Marks	Expected	Actual
1	dev	101	60	Pass	Pass ■

2	rev	406	59	Pass	Pass ■
3	fun	543	89	Pass	Pass ■

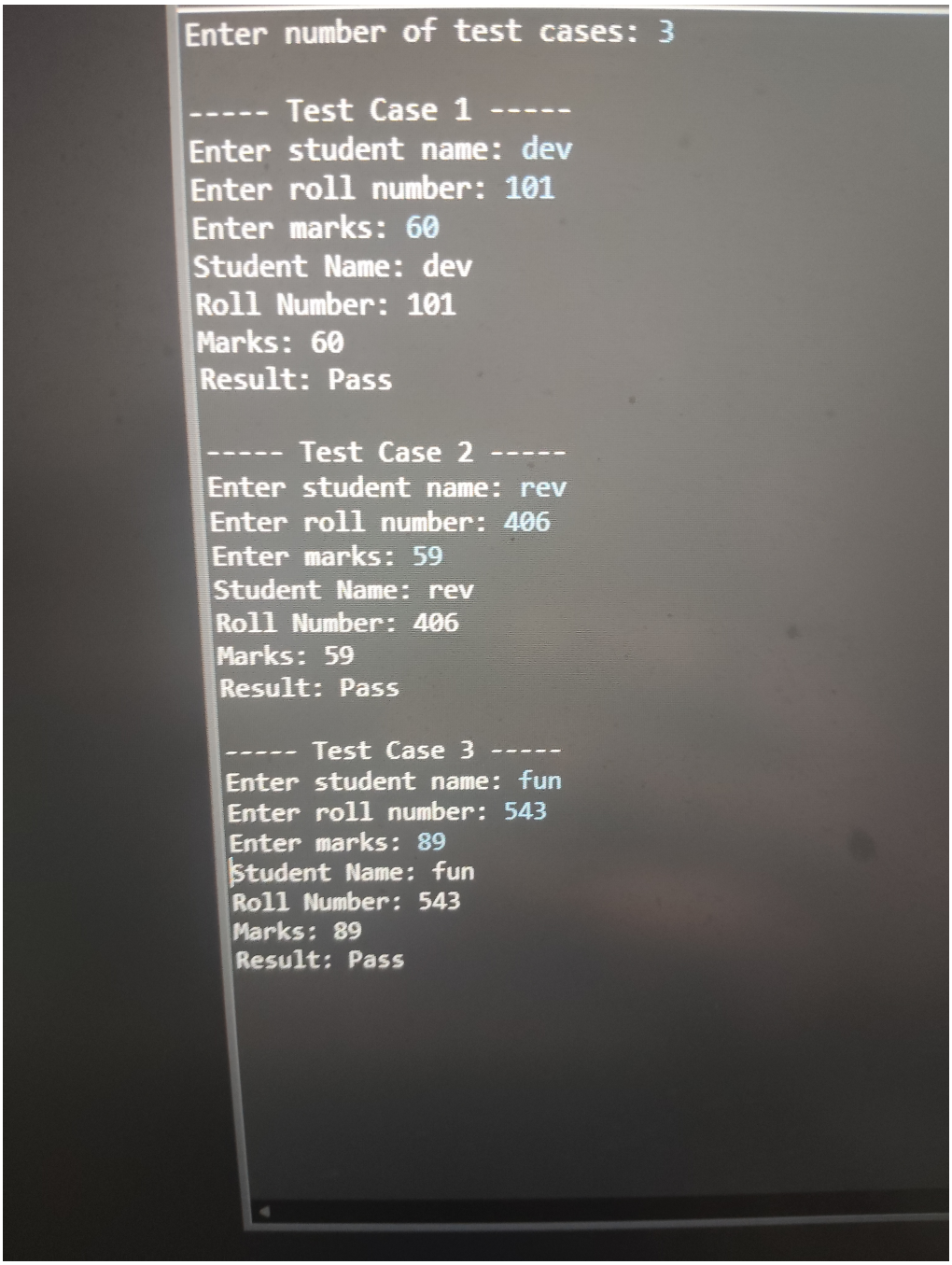
6. Inheritance Flow

```
       Student
      / name
     /  rollNo
    v
   Marks
  / marks
 v
 Result
 / displayResult()
```

Important: Result can access members/methods inherited from both Marks and Student. This is why it is called Multilevel Inheritance.

7. Output Screenshot

The screenshot below shows the program actually executed, running all three test cases in a single session:



The results verify that the program correctly propagates fields and methods across three levels of inheritance, confirming multilevel inheritance via chained `extends` works as expected.